INSTITUTO UNIVERSITÁRIO DE LISBOA

Mestrado em Engenharia Informática

Segurança em Redes e Sistemas de Informação

Segurança em Camadas para Comunicações IoT Baseadas em MQTT

Implementação, Avaliação num Raspberry Pi e Validação

Projeto — Parte 2: implementação e avaliação

Autores:

Rodrigo Gorjão, N.º 82972
João Afonso, N.º 97990
João Tendeiro, N.º 129087
Jossivalter Leal, N.º 146851

Docente: Daniel Maximino Caçador

Repositório: `https://github.com/joaoaeafonso/mqtt_secure`

Resumo

A primeira parte deste projeto propôs uma arquitetura de segurança em camadas para comunicações MQTT e um plano para a avaliar. Esta segunda parte apresenta o protótipo que a implementa, as medições feitas num Raspberry Pi 5 e a validação do protótipo perante os ataques previstos. Cada camada pode ser ligada e desligada de forma independente, o que permitiu medir o custo e o benefício de cada uma em doze configurações (K0 a K11).

[**Por fazer:** Três a quatro frases com os resultados principais, depois da campanha: custo do envelope face ao transporte; ataques impedidos por configuração; custo das assinaturas; configuração recomendada por caso de uso; veredicto das hipóteses.]

Palavras-chave: MQTT, Internet das Coisas, segurança em camadas, proteção fim-a-fim, avaliação de desempenho, Raspberry Pi.

Índice

Resumo **i**

1 Introdução **1**

2 Protótipo **1**
2.1 Visão geral . . . 1
2.2 O que foi implementado . . . 2
2.3 Decisões tomadas nesta parte . . . 2
2.4 Testes automatizados . . . 3

3 Método experimental **3**
3.1 Plataforma . . . 3
3.2 Configurações . . . 4
3.3 Métricas . . . 4
3.4 Ataques . . . 4
3.5 Procedimento . . . 5
3.6 Parâmetros fixados antes das medições . . . 5
3.7 Desvios ao plano . . . 6

4 Resultados: custo **7**
4.1 Custo por configuração (QP1) . . . 7
4.2 Custo marginal de cada camada . . . 7
4.3 Tamanho da mensagem e saturação (QP3) . . . 7
4.4 Aceleração criptográfica por *hardware* (QP4) . . . 7
4.5 Octetos e memória . . . 8
4.6 Plataforma de referência . . . 8

5 Resultados: proteção **8**
5.1 Ataques impedidos por configuração (QP2) . . . 8
5.2 Cada ataque contra a sua defesa, com controlo negativo . . . 8
5.3 Observações . . . 8

6 Critério de escolha e recomendação **9**

7 Hipóteses **9**

8 Validação do protótipo **9**
8.1 Conformidade com a especificação . . . 9
8.2 Validação do instrumento de medição . . . 10
8.3 Calibração da validade temporal . . . 10
8.4 Reinício do recetor . . . 11
8.5 Rotação e revogação . . . 11

9 Ameaças à validade e limitações **11**
9.1 Validade de construção 11
9.2 Validade interna 11
9.3 Validade externa 12
9.4 Limitações do protótipo 13

10 Conclusões **13**

Referências **14**

A Reprodução dos resultados **15**

1 Introdução

A primeira parte do projeto reviu a literatura sobre segurança em comunicações MQTT, propôs uma arquitetura em camadas (Parte 1, Secção 6) e definiu o plano para a avaliar (Parte 1, Secção 7). O objetivo desta segunda parte é o que esse plano enuncia: medir, num Raspberry Pi, quanto custa cada camada em desempenho e quanto protege, isoladamente e em combinação, para identificar a configuração com a maior proteção e o menor impacto no desempenho.

O documento responde às cinco perguntas experimentais da Parte 1:

QP1. Qual é o custo de cada camada num Raspberry Pi (processador, latência, débito, memória, octetos)? (Secção 4)

QP2. Que ataques impede cada camada, e quais só a combinação de camadas impede? (Secção 5)

QP3. Como varia o custo com o tamanho da mensagem e a carga, e quando satura o Raspberry Pi? (Secção 4)

QP4. Quanto pesa a aceleração criptográfica por *hardware*? (Secção 4)

QP5. Que configurações são ótimas de Pareto, e qual recomendar para cada caso de uso? (Secção 6)

Contributos desta parte.

- Um protótipo em C++23 da arquitetura proposta, com cada camada ativável de forma independente, e os seus testes automatizados (Secção 2).
- Um laboratório de carga que executa de forma automática as doze configurações, os ataques e os respetivos controlos negativos contra um *broker* real, e que guarda os dados brutos e o registo do ambiente de cada execução (Secção 3).
- As medições no Raspberry Pi 5 e num computador de secretária, a aplicação do critério de escolha e o veredicto das hipóteses H1 a H5 (Secções 4 a 7).
- A validação do protótipo e a calibração dos limites de validade temporal que a Parte 1 deixou em aberto (Secção 8).

O plano da Parte 1 foi seguido; o que foi concretizado de forma diferente, ou decidido só nesta parte, está declarado na Secção 3.7.

2 Protótipo

2.1 Visão geral

O protótipo tem três partes. A **biblioteca** (C++23) implementa as camadas como elementos de uma cadeia de processamento: cada mensagem que chega passa pela limitação de tráfego, pela camada de segurança (o envelope) e, opcionalmente, pela persistência; cada camada pode ser ligada e desligada em execução, sem reconstruir a cadeia. O ***gateway* de ensaio** liga essa

cadeia a um *broker* Mosquitto real através de um cliente Paho e reporta, por janela de medição, as mensagens entregues e rejeitadas (por motivo), a latência e o tempo de processador. O **laboratório de carga** (Python) lança o *broker*, o *gateway*, os sensores simulados e o atacante, aplica cada configuração e mede.

[**Por fazer:** Figura da arquitetura implementada (adaptar a da Parte 1, Secção 6.1, marcando o que corre em cada processo e em cada máquina).]

2.2 O que foi implementado

A Tabela 1 relaciona cada elemento da proposta com a sua concretização.

Tabela 1. Elementos da arquitetura proposta e a sua implementação.

Elemento (Parte 1)	Implementação	Onde
Envelope de 25 octetos; tópico e modo nos dados associados; modos de cifra, autenticação e assinatura (§6.4)	AES-256-GCM, GMAC e Ed25519 sobre OpenSSL; a mesma implementação em Python para os sensores, verificada pelos mesmos vetores de teste	`modules/security`, `msec_envelope.py`
ChaCha20-Poly1305 como alternativa (§6.4)	O algoritmo é um atributo da chave: o formato do envelope não muda e um recetor serve os dois	`chacha20_poly1305_cipher.cpp`
Ordem de verificação (§6.5)	Formato, modo aceite, validade, verificação criptográfica, chave–tópico, janela	`security_layer.cpp`
Validade temporal (§6.6)	Δ_a e Δ_s globais e por fluxo (filtro de tópico)	`FreshnessRule`
Janela anti-repetição, $w = 64$ (§6.6)	Por chave, em memória, com estado limitado	`replay_guard.cpp`
Restrição chave–tópico (§6.7)	Política com padrões por dono da chave, aplicada depois da autenticação	`key_policy.cpp`
Chaves por época, rotação e revogação (§6.8)	Rotação com período de graça; revogação de chaves, palavras-passe e certificados (CRL)	`pki.py`, `lab.py`
Limitação de tráfego antes da criptografia (§6.9)	*Token bucket* por tópico	`modules/limits`
Camada de transporte (§6.3)	TLS 1.3, autenticação mútua, palavras-passe, ACL por utilizador, identificador de cliente ligado ao utilizador, limites de pacote e de ligações	`broker.py` (Mosquitto)

2.3 Decisões tomadas nesta parte

Reinício do recetor. A Parte 1 registou como limitação que a janela anti-repetição se perde quando o recetor reinicia. A solução adotada não persiste a janela a cada mensagem: o recetor guarda de forma durável apenas a época de chaves mais recente que aceitou e, depois de um reinício, só aceita épocas posteriores. A gravação faz-se uma vez por rotação, antes de aceitar a primeira mensagem da nova época, e uma falha de leitura ou de escrita rejeita a mensagem.

Dependências entre mecanismos. A validade temporal, a anti-repetição e a restrição chave–tópico verificam campos do envelope. Uma configuração que as ligue com o envelope desligado é recusada, no arranque e em execução, em vez de as ignorar em silêncio.

Fora do âmbito. Não foram implementados um modo HMAC próprio (o GMAC cobre a autenticação sem cifra), *nonces* por contador nem o modo de cifra seguida de assinatura, que a

Parte 1 apresenta como possível extensão.

2.4 Testes automatizados

[**Por fazer:** Atualizar as contagens no fim.] O protótipo tem 93 testes em C++ (unitários, de integração e de aceitação), executados com AddressSanitizer e UndefinedBehaviorSanitizer, e 72 testes em Python (60 do laboratório, 12 da análise). Os testes de aceitação têm um teste por regra da ordem de verificação e foram verificados por mutação: cada um falha quando a regra correspondente é removida. Três alvos de *fuzzing* exercitam a análise do envelope, a validação de tópicos e todo o caminho de receção com octetos arbitrários. A integração contínua compila com `clang-tidy` (avisos tratados como erros) e corre os cenários de ataque contra um Mosquitto real.

3 Método experimental

3.1 Plataforma

A Tabela 2 descreve as duas máquinas. O Raspberry Pi 5 executa o *broker* e o *gateway*; o computador de secretária executa os sensores simulados (dois agentes) e o atacante (um agente). As duas máquinas estão ligadas por cabo, a 1 Gb/s, a um mesmo comutador Ethernet simples, sem router no caminho medido. No Raspberry Pi, o regulador de frequência estava em `performance`, o arrefecimento ativo oficial estava montado e os serviços alheios ao ensaio (incluindo o Docker) estavam parados durante a campanha.

Tabela 2. Plataforma do nível 2.

	Raspberry Pi 5 (*broker* e *gateway*)	**Computador de secretária** (sensores e atacante)
Processador	Raspberry Pi 5 Model B rev. 1.1 (4 núcleos Cortex-A76, 2,4 GHz)	AMD Ryzen 7 3700X (8 núcleos)
Memória	[**Por fazer:** RAM do Pi (`manifest.json`)]	32 GB
Armazenamento	SSD NVMe de 256 GB	–
Sistema	Debian (Raspberry Pi), *kernel* 6.18.60 (aarch64)	Ubuntu 24.04.5, *kernel* 7.0.0
Alimentação e arrefecimento	Fonte oficial de 27 W; arrefecimento ativo oficial	–
Rede	Ethernet 1 Gb/s	Ethernet 1 Gb/s
Software	GCC 14.2.0, CMake 3.31.6, Mosquitto 2.0.21, Python 3.13.5, OpenSSL [**Por fazer:** versão (`manifest.json`)]	Python 3.12.3

O código medido é o do *commit* [**Por fazer:** `manifest.json`], o mesmo nas duas máquinas: o controlador recusa agentes de outra versão.

Relógios. A latência é medida entre máquinas: o sensor escreve na mensagem o instante de envio e o *gateway* subtrai-o do instante de entrega. Cada agente estima o desvio do seu relógio em relação ao do Raspberry Pi por trocas de pedido e resposta no canal de controlo (duas por segundo) e os sensores corrigem o instante que escrevem. Dois relógios de quartzo independentes

afastam-se a um ritmo quase constante (nos pilotos, cerca de 13 ppm, ou seja, 0,8 ms por minuto, mais do que as latências a medir), pelo que o desvio não é o da última amostra: ajusta-se uma reta ao quarto das amostras do último minuto com menor tempo de ida e volta e avalia-se no instante presente. Os relógios foram sincronizados por NTP antes da campanha e o NTP foi desligado nas duas máquinas durante ela, porque o NTP altera o ritmo do relógio enquanto corrige um desvio, e essa variação não é uma reta.

Tal como planeado, a avaliação tem dois níveis. No **nível 1**, os *microbenchmarks* medem o tempo de processador da cadeia de camadas por mensagem, sem rede, para 16 B, 256 B, 4 KiB e 64 KiB. No **nível 2**, o *broker* e o *gateway* correm no Raspberry Pi e os sensores e o atacante noutras máquinas da rede local.

3.2 Configurações

Foram medidas as doze configurações da Parte 1, Tabela das configurações (K0 a K11), sem alterações. O nível 1 tem um perfil para K0, K2 a K7, K9 e K11; K1, K8 e K10 só diferem das vizinhas no transporte e nos limites, que só existem no nível 2.

3.3 Métricas

A Tabela 3 indica como cada métrica do plano foi medida.

Tabela 3. Como cada métrica foi medida.

Métrica	Medição
Processador por mensagem	Tempo de processador do processo do *gateway* na janela, dividido pelas mensagens recebidas (nível 2); tempo de processador por mensagem da cadeia completa (nível 1)
Latência p50 e p99	Do instante de envio, escrito pelo sensor na mensagem, à entrega depois da cadeia de camadas. O relógio de cada máquina de sensores é corrigido pelo desvio estimado em relação ao do *gateway*
Débito máximo sustentado	Maior taxa entregue com perda inferior a 1 % e p99 dentro de $L_{\max}$, numa subida de carga por configuração
Octetos por mensagem	Octetos recebidos na ligação TCP do *gateway* por mensagem, separados em conteúdo, envelope, MQTT e restante (TLS e controlo)
Memória	Pico de memória residente do *gateway* e do *broker* na janela
Estabelecimento de ligação	Tempo e tempo de processador de religações do *gateway* (TLS, CONNECT e subscrições), fora da janela de tráfego
Ataques impedidos, $\pi_a(c)$	Cada ataque contra cada configuração, com tráfego legítimo ativo; maioria sobre as repetições
Fuga de informação	Fração das mensagens legíveis por um subscritor de `#` e por um atacante na ligação do *gateway*

A energia, opcional no plano, não foi medida. [**Por fazer:** Confirmar, ou descrever o medidor se vier a ser usado.]

3.4 Ataques

O atacante do laboratório executa os ataques da Parte 1, Tabela dos ataques. Dois tipos de ensaio usam-nos de forma diferente:

- A **matriz de ataques** lança cada ataque contra cada configuração completa e dá $\pi_a(c)$, de

que resulta $S(c)$. Um ataque travado no *broker* (por exemplo, um estranho sem credenciais perante a autenticação mútua) conta como impedido por essa configuração.

- Os **cenários** isolam uma defesa de cada vez sobre uma base aberta e emparelham cada ataque com um *controlo negativo*, a mesma configuração sem essa defesa, em que o ataque tem de ter sucesso. Só assim se atribui uma rejeição à camada certa.

Os ataques internos partem de um dispositivo comprometido: o atacante recebe as chaves, a palavra-passe e o certificado de um sensor, e o sensor verdadeiro fica desligado enquanto o atacante o controla. Os ataques na rede usam um intermediário na ligação entre o *broker* e o *gateway*, que conta o que consegue ler e, em MQTT sem TLS, altera, redireciona, atrasa ou suprime mensagens, fazendo de *broker* malicioso.

3.5 Procedimento

Cada configuração foi repetida 10 vezes, em ordem aleatória com semente registada, com 3 s de estabilização e 30 s de medição por execução, a 1 000 mensagens/s (50 sensores a 20 mensagens/s cada).

O débito máximo foi medido por uma subida de carga, três vezes por configuração, em ordem aleatória: 10, 20, 50, 100, 125, 150, 200, 300, 400, 500, 600, 700, 800, 1 000, 1 200 e 1 600 mensagens/s por sensor (de 500 a 80 000 mensagens/s no total), 15 s cada, parando no primeiro patamar com perda de pelo menos 1 %. Os patamares são mais densos entre 100 e 800 porque é aí que os pilotos situaram os limites. Depois do patamar que satura, a carga volta ao mínimo e espera-se até o *gateway* receber apenas o que os sensores enviam: sem esta espera, as mensagens retidas na sobrecarga chegavam à configuração seguinte, que as rejeitava (outra chave, outro limite de débito) como se fossem tráfego dela.

A campanha foi precedida de três pilotos (quatro configurações, três repetições, 15 s), que serviram para fixar os patamares de carga e R_{alvo} e para validar o próprio instrumento: o primeiro revelou latências negativas (a deriva dos relógios, corrigida com a estimação descrita acima) e o segundo a contaminação entre patamares (corrigida com a espera). Os dados dos pilotos não são resultados e não entram nas tabelas. Os intervalos de confiança são de 95 %, por *bootstrap* de percentis sobre as execuções. O custo marginal de uma camada é a diferença de medianas entre as duas configurações que só diferem nela, e a diferença das diferenças mede a interação entre a validade temporal e a anti-repetição e entre o TLS e o envelope. A matriz de ataques correu separada das medições de custo, porque o intermediário na ligação do *gateway* acrescenta latência.

Cada campanha guarda os dados brutos de todas as execuções, a ordem dos ensaios, a semente, a configuração completa e um registo do ambiente (código, plataforma, versões, temperatura e frequência do processador por execução). Todas as tabelas e figuras deste relatório são geradas a partir desses dados por um único comando (Anexo A).

3.6 Parâmetros fixados antes das medições

Os parâmetros do critério de escolha foram fixados antes da campanha: o ficheiro que os contém é copiado para a pasta da campanha antes da primeira medição e o seu resumo criptográfico fica

no registo do ambiente.

- **Pesos dos ataques.** Cada linha da tabela de ataques da Parte 1 pesa 1. Duas linhas são medidas por dois ataques cada (a escuta, na rede e no *broker*; a injeção e a adulteração), que pesam 0,5 cada.
- **Pesos do custo.** Processador por mensagem 0,4; latência p99 0,4; débito máximo, octetos por mensagem, memória do *gateway* e estabelecimento de ligação 0,05 cada.
- **Exequibilidade.** O cenário de aplicação é a monitorização industrial: um *gateway* de periferia por área de uma instalação, que agrega 1 000 sensores de condição e vibração a 10 Hz e envia ordens a 100 atuadores, até 10 por segundo cada. Daí $R_{\text{alvo}} = 10\,000$ mensagens/s para a telemetria (os dois casos de uso) e $R_{\text{alvo}} = 1\,000$ mensagens/s para as ordens, com $L_{\max} = 50$ ms, um limite de tempo real não estrito adequado à supervisão e ao controlo. Os pilotos serviram apenas para confirmar que nenhum dos valores coincide com o limite de uma configuração, onde o veredicto mudaria de uma repetição para outra.

3.7 Desvios ao plano

- **Aceleração AES desligada.** O plano previa `OPENSSL_armcap=0`. Essa variável desliga todas as extensões ARM que o OpenSSL usa, incluindo o NEON, de que o ChaCha20-Poly1305 depende. Um dispositivo sem as extensões criptográficas tem, em geral, NEON; por isso mediu-se também com `OPENSSL_armcap=1` (só NEON), e é essa a comparação usada para a QP4. Os dois resultados são apresentados.
- **Estabelecimento de ligação.** O tempo de ligação medido varia em degraus de cerca de 100 ms, por esperas internas da biblioteca cliente, que escondem os poucos milissegundos do TLS. No custo agregado entra o tempo de processador por ligação; o tempo decorrido é reportado à parte.
- **Escuta no *broker*.** Na matriz de ataques, este ataque é um subscritor de `#` sem credenciais. Um *broker* curioso não é medido por $S(c)$ (Secção 9); é o requisito de confidencialidade do caso de uso que o cobre.
- **Dispositivo comprometido.** O sensor verdadeiro fica desligado durante o ataque. Com os dois ligados, disputam a sessão MQTT e o ataque fica reduzido a uma fração do tempo (Secção 5.3).
- **R_{alvo} por caso de uso.** O plano previa um só R_{alvo}. As ordens a atuadores são um fluxo muito menor do que a telemetria e só o modo de assinatura (K9) dá a prova de origem que exigem; com um só valor de 10 000 mensagens/s, que K9 não sustenta, esse caso de uso ficaria sem nenhuma configuração exequível por causa de um débito que as ordens nunca têm. Cada caso de uso tem por isso o seu R_{alvo}, fixado antes da campanha como os restantes parâmetros.
- **Instrumento corrigido depois dos pilotos.** A estimação do desvio dos relógios passou a corrigir a deriva e a subida de carga passou a esperar pelo escoamento das mensagens retidas (Secção 3, Procedimento). Não muda o que se mede, só a sua exatidão.

- [**Por fazer:** Outros desvios que surjam na campanha (repetições a mais, execuções repetidas, energia).]

4 Resultados: custo

4.1 Custo por configuração (QP1)

As Tabelas **??** e **??** dão, para cada configuração, a mediana e o intervalo de confiança de cada métrica no Raspberry Pi 5.

Em falta: `gerado/pi5/summary.tex`
(`make dados` depois da campanha)

Em falta: `gerado/pi5/fig-cpu.tex`
(`make dados` depois da campanha)

Figura 1. Tempo de processador do *gateway* por mensagem, por configuração (mediana e IC 95 %).

Em falta: `gerado/pi5/fig-p99.tex`
(`make dados` depois da campanha)

Figura 2. Latência p99 por configuração (mediana e IC 95 %).

[**Por fazer:** Leitura: que configurações se distinguem e quais ficam dentro do ruído; ordem de grandeza do custo do transporte e do envelope.]

4.2 Custo marginal de cada camada

Em falta: `gerado/pi5/marginal.tex`
(`make dados` depois da campanha)

[**Por fazer:** Comentar a Tabela **??** camada a camada. Interações (`interaction.csv`): a validade e a anti-repetição custam o mesmo juntas e separadas? O TLS custa o mesmo com e sem envelope?]

4.3 Tamanho da mensagem e saturação (QP3)

[**Por fazer:** Custo em função do tamanho: nível 1 (Tabela **??**) e, se medido, nível 2 com `-vary load.payload`. Débito máximo sustentado por configuração (`saturation.csv`): tabela ou figura, e o que limita primeiro (processador do *gateway*, do *broker* ou a rede).]

4.4 Aceleração criptográfica por *hardware* (QP4)

Em falta: `gerado/pi5/microbench.tex`
(`make dados` depois da campanha)

[**Por fazer:** AES-GCM e ChaCha20-Poly1305 com a aceleração ligada, com `OPENSSL_armcap=1` (só NEON) e com `OPENSSL_armcap=0`; K8 contra K11 no nível 2. Em que condições o ChaCha20 compensa.]

4.5 Octetos e memória

[**Por fazer:** Octetos por mensagem: confirmar os 53 e 89 octetos do envelope e quanto acrescenta o TLS por mensagem. Memória do *gateway* e do *broker*.]

4.6 Plataforma de referência

[**Por fazer:** O mesmo no computador de secretária (Anexo A): em que é que a ordenação das configurações muda, se mudar.]

5 Resultados: proteção

5.1 Ataques impedidos por configuração (QP2)

[**Por fazer:** Tabela $\pi_a(c)$: configurações em linhas, ataques em colunas, a partir de `gerado/pi5/protection.csv`, com $S(c)$ na última coluna. Indicar o número de repetições e os casos em que as repetições não concordaram.]

[**Por fazer:** Leitura: o que K1 já impede (tudo o que exige chegar ao *broker*); o que só o envelope impede (K10 sem TLS, *broker* ativo); o que só a combinação impede (mensagem antiga nunca recebida: validade; repetição: janela); o que só K8 e seguintes impedem (inundação, ocupação de ligações).]

5.2 Cada ataque contra a sua defesa, com controlo negativo

[**Por fazer:** Tabela dos cenários (`results/<campanha>/scenarios.md`): ataque, defesa, previsão, resultado e evidência, com o controlo negativo ao lado. Esta tabela confirma ou refuta, linha a linha, a tabela de ataques da Parte 1.]

5.3 Observações

Os ensaios mostraram efeitos que a tabela de ataques da Parte 1 não previa. [**Por fazer:** Confirmar cada um com os dados do Raspberry Pi e quantificar.]

- **Ocupação de ligações.** O limite de ligações protege os recursos do *broker* e mantém os clientes já ligados, mas o atacante ocupa as vagas livres; se o *broker* reiniciar durante o ataque, o atacante pode ficar com todas as vagas e nenhum sensor volta a ligar-se.
- **Chave roubada e janela da vítima.** As mensagens aceites com uma chave roubada avançam a janela anti-repetição dessa chave; o tráfego legítimo do dispositivo passa a ser rejeitado até à rotação de chaves.

- **Credencial clonada e sessão MQTT.** Se o atacante se liga com a identidade de um dispositivo que continua ligado, o *broker* entrega-lhe a sessão e desliga o legítimo; os dois alternam enquanto o ataque dura, o que é uma negação de serviço da vítima.
- **Falsificação pelo recetor num *broker* legítimo.** A ACL já impede um atuador de publicar nos tópicos de ordens dos outros; a diferença entre a chave partilhada e a assinatura só se observa sem ACL ou com um *broker* ativo.
- **Atraso e supressão.** Um *broker* ativo que atrase mensagens menos do que Δ_a, ou que as suprima, não é detetado pelo recetor.

6 Critério de escolha e recomendação

Aplica-se o critério da Parte 1, Secção 7.5 com os parâmetros da Secção 3.6: proteção $S(c)$, custo normalizado por K0, exequibilidade, requisitos por caso de uso e fronteira de Pareto.

Em falta: `gerado/pi5/criterion.tex`
(`make dados` depois da campanha)

Em falta: `gerado/pi5/fig-pareto.tex`
(`make dados` depois da campanha)

Figura 3. Proteção $S(c)$ contra custo(c); a tracejado, a fronteira de Pareto para a telemetria sensível.

[**Por fazer:** Para cada caso de uso (telemetria não sensível, telemetria sensível, ordens a atuadores): configurações admissíveis, fronteira de Pareto, c^* e custo marginal $\Delta\,\text{custo}/\Delta S$ ao longo da fronteira. Confirmar se a recomendação antecipada na Parte 1 (K8, ou K11 sem AES em *hardware*, para a telemetria; K9 para as ordens) se mantém. Sensibilidade: a recomendação muda com pesos diferentes?]

7 Hipóteses

A evidência de cada hipótese está em `gerado/pi5/hypotheses.md`; o veredicto é dos autores.

[**Por fazer:** Um parágrafo por hipótese refutada ou só em parte confirmada. A Parte 1 diz que as hipóteses podem ser refutadas: dizê-lo claramente quando acontecer.]

8 Validação do protótipo

8.1 Conformidade com a especificação

A validação tem três níveis. Os **testes automatizados** (Secção 2) verificam cada regra de aceitação, os vetores de teste do envelope (partilhados pela implementação em C++ e pela em Python, que assim não podem divergir) e os vetores do RFC 8439 para o ChaCha20-Poly1305. Os **cenários** (Secção 5) verificam o sistema completo contra um *broker* real: os 39 cenários deram o resultado previsto no Raspberry Pi, antes da campanha (`results/validation/pi5-20261111/`).

Tabela 4. Veredicto das hipóteses da Parte 1.

Hipótese	Evidência	Veredicto
H1. O envelope AES-256-GCM tem um custo por mensagem pequeno face à latência da rede	[**Por fazer:** K1→K3: processador e p50]	[**Por fazer:**]
H2. Validade, anti-repetição e chave–tópico têm custo marginal quase nulo	[**Por fazer:** K3→K4, K3→K5, K6→K7]	[**Por fazer:**]
H3. Ed25519 custa claramente mais do que AES-GCM por mensagem	[**Por fazer:** K8→K9]	[**Por fazer:**]
H4. O custo do TLS concentra-se no estabelecimento da ligação	[**Por fazer:** processador por ligação K0 e K1; K0→K1 e K10→K7 por mensagem]	[**Por fazer:**]
H5. Sob inundação, a limitação antes da criptografia reduz o processador do *gateway*	[**Por fazer:** ataque de inundação em K7 e K8]	[**Por fazer:**]

No mesmo *commit* da campanha, os 93 testes em C++ e os 60 testes em Python do laboratório passaram no Raspberry Pi ([**Por fazer:** confirmar `tests-campaign.txt` no *commit* final]); só os sete testes que dependem do `tshark` não correram, por o Wireshark não estar instalado no Raspberry Pi. A **matriz de ataques** confirma que cada configuração impede o que o desenho prevê.

8.2 Validação do instrumento de medição

Os pilotos no Raspberry Pi validaram também o próprio laboratório, e revelaram dois defeitos que teriam falseado a campanha:

- **Deriva dos relógios.** No primeiro piloto, a latência mediana de K0 saiu negativa (-455 μs) e variava de 300 a 1 055 μs entre repetições da mesma configuração. O desvio entre os relógios crescia de forma regular (de 32,9 para 51,8 ms ao longo da subida de carga, cerca de 13 ppm), e a estimação usava a melhor amostra do último minuto, que podia estar 0,8 ms atrasada. Com a reta ajustada às amostras (Secção 3), a latência mediana de K0 ficou entre 132 e 133 μs nas três repetições do terceiro piloto. Um teste reproduz a deriva observada e exige um erro inferior a 50 μs.
- **Contaminação entre patamares.** No segundo piloto, uma execução de K8 no primeiro patamar (500 mensagens/s) registou 77 388 rejeições e p99 de 1,3 s: seguia-se a um patamar de K11 em sobrecarga, cujas mensagens retidas chegaram depois da mudança de chave e de limite de débito. Se a perda tivesse passado de 1 %, a subida de carga de K8 teria parado logo nesse patamar e o seu débito máximo ficaria por medir. Com a espera pelo escoamento, nenhuma execução posterior a uma saturação teve rejeições no terceiro piloto.

8.3 Calibração da validade temporal

A Parte 1 fixou valores iniciais, $\Delta_a = 60$ s e $\Delta_s = 5$ s, a calibrar nesta parte a partir da latência medida e do desvio de relógio observado. Cada execução regista a cauda da latência e o desvio entre o relógio de cada máquina de sensores e o do *gateway*.

[**Por fazer:** Copiar de `gerado/pi5/calibration.md`: latência máxima, desvio de relógio máximo e incerteza residual; valores propostos de Δ_a e Δ_s com relógios sincronizados e sem sincronização (margem de segurança 2, arredondamento a 100 ms). Comentar o que isto significa para as ordens a atuadores, que justificam uma validade mais curta por fluxo.]

8.4 Reinício do recetor

[**Por fazer:** Demonstração: reiniciar o *gateway*; as mensagens com chaves da época anterior são rejeitadas (incluindo repetições capturadas antes do reinício); depois da rotação, o tráfego retoma. Indicar o tempo sem serviço.]

8.5 Rotação e revogação

Os dois cenários passaram no Raspberry Pi. Em `key-rotation`, todas as chaves do conteúdo são substituídas com o tráfego a correr, sem nenhuma rejeição e sem perda (0,0 %). Em `revocation`, o sensor 1 é revogado (chaves e palavra-passe): fica de fora e os outros nove continuam, com o débito entregue a descer de 200 para 180 mensagens/s, exatamente a parte do sensor revogado. [**Por fazer:** Confirmar com os cenários da campanha (`results/<campanha>/scenarios`).]

9 Ameaças à validade e limitações

9.1 Validade de construção

- $S(c)$ **conta ataques, não a sua gravidade.** Com os pesos usados, todos os ataques valem o mesmo; a exclusão por caso de uso compensa em parte, porque uma propriedade obrigatória em falta não é trocada por custo.
- **A escuta no *broker* mede um subscritor estranho.** Na matriz de ataques, quem escuta é um cliente sem credenciais, que K1 já impede. Um *broker* curioso ou comprometido lê o conteúdo em K1, K2 e K9, que não cifram perante o *broker*, e $S(c)$ não o mostra. Quem o mostra é o filtro por caso de uso (a telemetria sensível exige o modo de cifra) e o cenário que compara o que um subscritor lê com TLS e com o envelope.
- **Um ataque travado no *broker* conta como impedido.** As configurações com transporte protegido impedem muitos ataques por o atacante não chegar ao *broker*; isso deixa de valer se o *broker* for o adversário. K10 e os cenários com *broker* ativo medem o que resta sem essa barreira.
- ***Broker* ativo só em MQTT sem TLS.** O intermediário não termina TLS; um *broker* malicioso real fá-lo-ia. O que o envelope deteta não depende do transporte, pelo que o resultado se mantém, mas não foi medido com TLS.

9.2 Validade interna

- **O instrumento pesa na medição.** O tempo de processador do *gateway* inclui a contabilidade das medições, e a sua memória inclui as amostras de latência da janela e a tabela de mensagens

vistas, que cresce nos primeiros minutos de cada laboratório. As diferenças entre configurações são válidas; os valores absolutos de memória são um majorante.

- **Tempo de ligação.** Varia em degraus de cerca de 100 ms por esperas internas da biblioteca cliente; usa-se o tempo de processador por ligação.
- **Relógios.** A latência entre máquinas depende da correção do relógio dos sensores, com incerteza igual a metade do melhor tempo de ida e volta observado, entre 0,13 e 0,21 ms nos pilotos. É da ordem das diferenças de latência mediana entre configurações vizinhas (K0 133 μs, K8 188 μs no terceiro piloto), mas a mesma para todas: como a ordem é aleatória, desloca as medidas sem favorecer nenhuma configuração. A deriva é corrigida (Secção 3); uma alteração do ritmo do relógio a meio de uma execução não o é, e por isso o NTP esteve desligado durante a campanha. [**Por fazer:** Confirmar com `clock_uncertainty_ms_max` e `clock_offset_ms_max` da campanha.]
- **Temperatura e frequência.** Nos três pilotos não houve *throttling* (`throttled` sempre `0x0`), a frequência esteve sempre em 2 400 MHz e a temperatura entre 44 e 52 °C; a ordem aleatória distribui o efeito que houver pelas configurações. [**Por fazer:** Confirmar com a campanha.]
- **Sensores simulados.** Os sensores são processos Python num computador, não dispositivos restritos: o custo no emissor não foi medido no nível 2 (o nível 1 mede envio e receção).
- **Uma só máquina de carga.** Os dois agentes de sensores e o atacante correm no mesmo computador. Ele gerou 80 000 mensagens/s nos pilotos sem ficar aquém da taxa pedida, mas o que acontece nele (escalonamento, recolha de memória do Python) entra na latência medida.
- **Picos de latência.** Em todas as configurações, incluindo K0, que não tem criptografia, surgem nos pilotos picos de 22 a 40 ms no p99,9 e no máximo, a cargas intermédias. Por existirem em K0, vêm do ambiente e não das camadas medidas; não afetam o p99 abaixo de cerca de 300 mensagens/s por sensor. A partir de 400 mensagens/s por sensor (20 000 mensagens/s), a latência mediana sobe de cerca de 0,2 para 8–14 ms em todas as configurações, com o *gateway* entre 40 e 60 % de processador, longe da saturação. [**Por fazer:** Identificar a origem na campanha (agentes ou *broker*).]
- **Serviços alheios.** O Docker esteve ativo no Raspberry Pi durante o teste de fumo e os pilotos, e foi parado para a campanha; os pilotos só serviram para fixar parâmetros, não entram nos resultados.

9.3 Validade externa

- Um só modelo de *hardware* restrito (Raspberry Pi 5), um só *broker* (Mosquitto) e uma rede local. Um microcontrolador sem sistema operativo, ou uma rede com perdas, podem ordenar as configurações de outra forma.
- **Sem AES em *hardware*** foi emulado desligando extensões no OpenSSL do mesmo processador, o que não reproduz a memória nem a *cache* de um processador mais simples.

- Carga sintética: mensagens de tamanho fixo a taxa constante, QoS 0. [**Por fazer:** Ajustar ao que foi medido.]

9.4 Limitações do protótipo

As limitações da Parte 1, Secção 6.11 mantêm-se, com uma alteração: o reinício do recetor deixou de reabrir a janela anti-repetição (Secção 2), à custa de exigir uma rotação de chaves depois de cada reinício. Continuam por resolver a distribuição de chaves fora do laboratório, o armazenamento de chaves (só em memória), a supressão e o atraso de mensagens por um *broker* ativo, e a confidencialidade e a prova de origem em simultâneo (cifra seguida de assinatura).

10 Conclusões

[**Por fazer:** Responder a cada pergunta numa ou duas frases, com números: QP1 (custo de cada camada), QP2 (o que cada camada impede), QP3 (tamanho e saturação), QP4 (aceleração por *hardware*), QP5 (configuração recomendada por caso de uso).]

[**Por fazer:** O que o trabalho acrescenta à lacuna L5 da Parte 1 (falta de medições do custo e do benefício de cada camada, isoladamente e em combinação, em *hardware* restrito).]

[**Por fazer:** Trabalho futuro: cifra seguida de assinatura; distribuição de chaves; medição no emissor em *hardware* restrito; *broker* ativo com TLS; energia.]

Referências

A Reprodução dos resultados

O repositório contém o código, os dados brutos de cada campanha (`results/`) e os programas de análise. Os passos para repetir as medições num Raspberry Pi estão em `docs/raspberry-pi.md`; o procedimento e o significado de cada ficheiro, em `docs/evaluation.md`.

- **Refazer as tabelas e figuras a partir dos dados brutos:** `tools/analysis/analyze.py results/<campanha>` e, nesta pasta, `make dados PI=<campanha>` seguido de `make`.
- **Repetir as medições:** `loadgen.py campaign` com a mesma semente e o mesmo `params.toml` (guardados em cada campanha).

[**Por fazer:** Tabela com as campanhas usadas: nome, plataforma, data, *commit*, semente, número de execuções.]

Plataforma de referência

Em falta: `gerado/desktop/summary.tex`
(`make dados` depois da campanha)

Em falta: `gerado/desktop/criterion.tex`
(`make dados` depois da campanha)